

Wie viel Wahrnehmung braucht ein NEAT-Agent?

Einfluss von Zielrichtung und Sichtradius auf das Lösen von Labyrinthen

Tobias Bandt
HAW Hamburg
Hamburg, Deutschland
eawp@tobiasbandt.de

Abstract

Abstract-Text here

Keywords

Neuroevolution, NEAT, Labyrinth, Wahrnehmungsradius, Evolutionäre Algorithmen

1 Einführung

Im Sommersemester 2026 wurde an der Hochschule für Angewandte Wissenschaften in Hamburg der Wahlpflichtkurs Evolutionäre Algorithmen von Prof. Dr.-Ing. Christian Lins angeboten. Im Verlauf dieses Kurses wurden Themen wie Hill Climbing, Differential Evolution, Genetische Algorithmen und NEAT in praktischen Übungen vertieft. Zum Kurs gehört ein Praktikum aus sechs aufeinander aufbauenden Aufgabenblättern diese Ausarbeitung knüpft an das dritte an, in dem mit NEAT ein neuronales Netz evolviert wird, das einen Agenten durch ein zufällig erzeugtes Labyrinth steuert.

Bei der Bearbeitung ist mir aufgefallen, dass der Agent zwar lernt, nicht gegen Wände zu laufen, das Ziel aber nur selten und eher zufällig erreicht. Der Grund liegt in den Eingaben: Die acht Umgebungsfelder sagen dem Netz, wohin es sich bewegen *kann*, aber nicht, wohin es sich bewegen *sollte*. Ein vorwärtsgerichtetes Netz bildet dieselbe Eingabe immer auf dieselbe Ausgabe ab, also führen zwei lokal gleich aussehende Stellen im Labyrinth zwangsläufig zur selben Bewegungsrichtung – unabhängig davon, wo das Ziel liegt. Das Aufgabenblatt nennt die acht Felder allerdings nur als Untergrenze und stellt ausdrücklich frei, dem Agenten „vielleicht etwas mehr“ zu zeigen.

Daraus ergibt sich die Fragestellung: Wie viel Wahrnehmung – in Form von Zielrichtung und Sichtradius – braucht der Agent, um ein Labyrinth zuverlässig zu lösen, und ab wann schadet mehr Eingabe mehr, als sie nützt? Der zweite Teil ist dabei genauso wichtig wie der erste, denn jede zusätzliche Eingabe vergrößert das Netz und damit den Suchraum, den NEAT durchsuchen muss. Meine Vermutung ist deshalb ein „Sweet Spot“. Mehr Wahrnehmung hilft bis zu einem Punkt und wird danach zum Nachteil. Ergänzend werden einige NEAT-Parameter variiert. Kapitel 2 beschreibt die Aufgabe und den Stand des Codes, Kapitel 3 die Varianten und die vorab festgelegten Hypothesen, die Kapitel 4 bis 7 Umsetzung, Messwerte und Bewertung.

2 Ausgangszustand

2.1 Aufgabenstellung und Umsetzung

Das Aufgabenblatt verlangt, mit `neat-python` Individuen mit mindestens acht Eingängen (die acht Umgebungsfelder) und vier Ausgängen (die vier Richtungen) zu evolvierten, eine Fitness-Funktion

zu schreiben, die den Agenten schrittweise durch eine zufällig erzeugte Karte steuert, und Pfad sowie bestes Netz zu visualisieren. Der Agent hat ein festes Schrittbudget und bekommt die beste Bewertung, wenn er das Ziel erreicht andernfalls muss die Fitness sinnvoll skaliert werden, etwa über den Abstand zum Ziel oder die Anzahl erfolgreicher Schritte. Kartengröße, Start- und Zielpunkt sind frei wählbar, und das Blatt fordert explizit zum Experimentieren mit Zielfunktionen und Kartenparametern auf.

Meine Umsetzung baut auf der mitgelieferten Vorlage `agent-neat-vorlage.py` auf. Die Klasse `MapGenerator` erzeugt eine 25×25-Karte, in der jedes Feld mit 25 % Wahrscheinlichkeit eine Wand ist. Der Agent startet unten rechts und soll das Ziel oben links erreichen, gezeichnet wird über ein `Tkinter-Canvas`. Zwei Ungenauigkeiten der Vorlage seien erwähnt, weil sie beim Lesen des Codes irritieren: Die Marker 'S' und 'E' sind gegenüber der Agentensicht vertauscht, und die Richtungsnamen in `move()` passen nicht zur Zeichenrichtung des Canvas. Beides ist rein kosmetisch, da das Netz seine Ausgänge nur über deren Wirkung lernt.

`move()` prüft, ob das Zielfeld innerhalb der Karte und keine Wand ist. Ungültige Schritte lassen den Agenten stehen, verbrauchen aber trotzdem einen Schritt. Zusätzlich habe ich einen Schalter für Backtracking eingebaut. Aktiv ist die Variante ohne: Besuchte Felder gelten als gesperrt, damit ein Netz nicht einfach zwischen zwei Feldern pendelt. Im Gegenzug ist eine Sackgasse endgültig, denn ab dort ist jeder weitere Schritt ungültig.

2.2 Wahrnehmung und Fitness

Die Wahrnehmung steckt vollständig in `_get_map_env()` (Listing 1). Die Methode liest die acht Nachbarfelder aus, wertet Felder außerhalb der Karte als Wand und gibt einen Vektor der Länge acht zurück, passend zu `num_inputs = 8`.

```
1 def _get_map_env(self):
2     env = []
3     def get_value(x, y):
4         if x < 0 or y < 0 or x >= len(self.map) \
5             or y >= len(self.map[0]):
6             return 1 # ausserhalb der Karte = Wand
7         elif self.map[x][y] in ('E', 'S'):
8             return 0
9         return self.map[x][y]
10    # alle acht Nachbarfelder, hier verkuerzt
11    for dx, dy in [(-1,1),(0,1),(1,1),(-1,0),
12                  (1,0),(-1,-1),(0,-1),(1,-1)]:
13        env.append(get_value(self.pos_x+dx, self.pos_y+dy))
14    return env
```

Listing 1: Wahrnehmung im Ausgangszustand: acht Nachbarfelder, keine Zielinformation.

Diese Methode ist die zentrale Stellschraube dieser Arbeit: Sowohl ein größerer Sichtradius als auch eine Zielrichtung lassen sich hier einbauen, ohne den Rest des Programms zu ändern – lediglich

Tabelle 1: Wichtigste Parameter aus neat-config im Ausgangszustand.

pop_size	1000	num_inputs / num_outputs	8 / 4
fitness_threshold	40,0	num_hidden	2
feed_forward	False	activation	relu
conn_add_prob	0,5	conn_delete_prob	0,2
node_add_prob	0,3	node_delete_prob	0,1
compatibility_threshold	2,8	weight_mutate_rate	0,8
		elitism	2

num_inputs muss zur neuen Vektorlänge passen. Genauso wichtig ist, was nicht darin steht: keine Position, kein Abstand, keine Richtung zum Ziel.

Die Methode run() führt pro Agent bis zu MAX_STEPS = 156 Schritte aus (bei 100 Schritten bliebe kaum Spielraum, da der kürzeste Weg schon 48 Schritte lang ist) und berechnet dabei die Fitness:

$$F = (d_{\text{start}} - d_{\text{best}}) + 10 \cdot [\text{Ziel erreicht}] + \sum_t (0,1 \cdot [\text{Schritt gültig}] - 0,1) \quad (1)$$

Der erste Term belohnt nicht den Abstand am Ende, sondern die größte je erreichte Annäherung, sodass ein Agent seinen Fortschritt nicht verliert, wenn er später in eine Sackgasse läuft. Der Bonus von 10 hebt erfolgreiche Agenten deutlich ab, und die Schrittbilanz bestraft effektiv nur ungültige Schritte. Bei einem Startabstand von $\sqrt{24^2 + 24^2} \approx 33,94$ liegt die maximal erreichbare Fitness bei rund 43,94, die konfigurierte fitness_threshold von 40,0 ist also ein brauchbares Abbruchkriterium für „gelöst“.

2.3 NEAT-Konfiguration und Schwachstellen

eval_genomes() baut aus jedem Genom ein Netz, lässt einen frischen Agenten laufen und schreibt dessen Fitness zurück. Alle Genome werden auf derselben Karte bewertet, da der Aufruf zum Neuerzeugen pro Generation auskommentiert ist. Das macht die Läufe vergleichbar, erlaubt einem Netz aber prinzipiell, die Karte auswendig zu lernen. Tabelle 1 zeigt die wichtigsten Parameter aus neat-config. Gerechnet wird mit 1000 Individuen über maximal 100 Generationen, mit vorzeitigem Abbruch beim Überschreiten der Fitness-Schwelle.

Damit ergeben sich vier Schwachstellen. Erstens fehlt jede Zielinformation, sodass lokal identische Situationen zwangsläufig zur selben Entscheidung führen. Zweitens erkennt der Agent bei einem Sichtradius von einem Feld eine Sackgasse erst, wenn er schon darin steht. Drittens verfällt ohne Backtracking nach dem Festfahren das restliche Schrittbudget. Viertens steht feed_forward auf False, erlaubt also rekurrente Verbindungen, während der Code die Netze mit FeedForwardNetwork.create() aufbaut – solche Verbindungen gehen beim Netzaufbau verloren, ein Teil der Mutationen läuft also ins Leere. Die ersten beiden Punkte sind die Stellschrauben dieser Arbeit. Der dritte wird für alle Varianten gleich gehalten, der vierte dagegen ist schlicht ein Fehler in der Konfiguration und wird vor der ersten Messung behoben (Kapitel 3).

Tabelle 2: Untersuchte Varianten. num_inputs muss zur Länge des Eingabevektors passen.

Variante	Beschreibung	r	Ziel	num_inputs
V0	Referenz (Ausgangszustand)	1	nein	8
V1	nur Zielrichtung	1	ja	10
V2	Radius 2 ohne Ziel	2	nein	24
V3	Radius 2 mit Ziel	2	ja	26
V4	Radius 3 mit Ziel	3	ja	50

3 Vorgehen

3.1 Varianten

Das Vorgehen folgt einem klassischen Versuchsaufbau: eine Referenz und darauf aufbauend Varianten, bei denen jeweils genau eine Stellschraube verändert wird. Die Hypothesen habe ich bewusst vor der ersten Messung notiert, weil sich Messergebnisse im Nachhinein fast immer irgendwie erklären lassen. Zwei Dinge werden vorab geklärt, die selbst nicht untersucht werden. Zum einen wird die in Kapitel 2 beschriebene Unstimmigkeit korrigiert und feed_forward auf True gesetzt, passend zum verwendeten Netztyp – ohne diese Korrektur wären besonders die Aussagen zu den Mutationsraten nicht interpretierbar. Zum anderen bleibt Backtracking für alle Varianten ausgeschaltet, wird also konstant gehalten statt korrigiert. Beides gilt auch für die Referenz V0, damit die Vergleichbarkeit erhalten bleibt.

Der Sichtradius r bezeichnet die Anzahl der Felder, die der Agent in jede Richtung sieht, was $(2r + 1)^2 - 1$ Umgebungsfeldern entspricht: 8 Felder bei $r = 1$, 24 bei $r = 2$ und 48 bei $r = 3$. Die Zielrichtung kommt als zwei auf $[-1, 1]$ normierte Eingaben Δx und Δy hinzu, jeweils die Differenz der Koordinaten geteilt durch MAP_SIZE - 1. Die Normierung sorgt dafür, dass die Zielrichtung dieselbe Größenordnung hat wie die Wandeingaben und nicht allein durch ihren Zahlenwert dominiert. Tabelle 2 zeigt die Varianten. V2 ist die Kontrolle zu V1: Sie vergrößert die Eingabe, ohne die Zielrichtung zu verraten, und zeigt damit, ob die Art oder nur die Menge der Information hilft.

3.2 Hypothesen

H1: Die Zielrichtung ist der entscheidende Faktor. V1 löst das Labyrinth deutlich zuverlässiger als V0 und braucht weniger Generationen, weil das Netz erst mit Δx und Δy zwischen lokal gleich aussehenden Situationen unterscheiden und eine Strategie der Form „gehe Richtung Ziel, weiche an Wänden aus“ überhaupt abbilden kann.

H2: Mehr Eingaben allein bringen wenig. V2 verbessert die Erfolgsquote gegenüber V0 nur geringfügig und bleibt hinter V1 zurück, obwohl V2 mehr Werte bekommt. Ein größerer Radius liefert mehr Details zur Wandstruktur, aber weiterhin keinen Hinweis auf die Zielrichtung.

H3: Der Nutzen zusätzlicher Wahrnehmung ist nicht monoton. V3 ist besser als V1, V4 dagegen wieder schlechter als V3. Radius 2 hilft, Sackgassen zu erkennen, bevor der Agent hineinläuft – ein direkter Vorteil, weil sie ohne Backtracking endgültig sind. Bei Radius 3 wächst der Eingabevektor auf 50 Werte, NEAT muss

aus deutlich mehr möglichen Verbindungen die wenigen relevanten herausfiltern, und innerhalb desselben Generationenbudgets erwarte ich ein schlechteres Ergebnis. Das Maximum vermute ich bei $r = 2$.

H4: Die NEAT-Parameter wirken schwächer als die Wahrnehmung. Populationsgröße, `compatibility_threshold` sowie `conn_add_prob` und `node_add_prob` verschieben die Ergebnisse messbar, aber in kleinerem Umfang als der Wechsel von V0 auf V1, weil Parameter nur steuern, wie schnell und breit gesucht wird, und keine Strategie ermöglichen können, die sich mit den vorhandenen Eingaben nicht darstellen lässt.

3.3 Messgrößen und Versuchsaufbau

Die Parameterstudie zu H4 läuft nicht auf allen Varianten, sondern nur auf der besten Wahrnehmungsvariante, und variiert jeweils einen Parameter um seinen Ausgangswert herum (`pop_size` 250/500/1000, `compatibility_threshold` 2,0/2,8/3,5, `conn_add_prob` 0,3/0,5/0,7, `node_add_prob` 0,1/0,3/0,5). Eine vollständige Kombination aller Varianten mit allen Werten wäre bei 1000 Individuen und bis zu 100 Generationen zeitlich nicht sinnvoll durchführbar.

Erfasst werden pro Lauf die *Erfolgsquote* (Anteil der Wiederholungen, in denen das Ziel innerhalb von 100 Generationen erreicht wurde), die *Generationen bis zur Lösung*, die *beste Fitness* – als einzige Kennzahl auch für erfolglose Läufe aussagekräftig – und die *Schritte bis zum Ziel* des besten Agenten als Maß dafür, ob eine Lösung effizient oder eher zufällig zustande kam.

Jede Variante wird mit $N = 10$ Wiederholungen gerechnet, da NEAT durch Startpopulation, Mutation und Elternauswahl stochastisch ist und ein einzelner Lauf nichts belegt. Alle Varianten verwenden dieselbe Folge protokollierter Seeds und dieselbe feste Karte, sodass keine Variante von einem leichteren Labyrinth profitiert und sich alle Läufe reproduzieren lassen. Dieser Aufbau prüft allerdings keine Verallgemeinerung: Ein Netz darf die Karte auswendig lernen. Lösen mehrere Varianten die feste Karte zuverlässig und lassen sich dadurch nicht mehr unterscheiden, wird als Erweiterung ein Durchgang mit pro Generation neu erzeugtem Labyrinth gerechnet. Ausgewertet wird je Variante über Mittelwert und Streuung der Messgrößen sowie über den Fitness-Verlauf pro Generation.

4 Durchführung

Code-Diffs, code erklären, Metriken, Seeds/Mazes.

5 Ergebnisse

Nur Fakten/Messwerte, Diagramme pro Konfiguration.

6 Diskussion

Interpretation, Hypothesen bestätigt/verworfen, eigene Einschätzung.

7 Fazit

Rückblick auf das Projekt.